TCC805x MCU BSP

API Specification

for Inter-Integrated Circuit Master Controller

Rev. 1.00 [A]

2020-11-12

TABLE OF CONTENTS

Contents

Figures

Tables

1 INTRODUCTION

This document describes the inter-integrated circuit master controller device driver (hereinafter referred to as I2C driver) of TCC805x MCU BSP. For more detailed information of the inter-integrated circuit master controller in the TCC805x MCU Sub-System, refer to Chapter 13 Inter-Integrated Circuit (I2C) Master Controller of *"PART 11-MICOM SUB-SYSTEM"* in the *"TCC805x Full Specification"*. [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

I2C	Inter-Integrated Circuit
SCL	I2C Clock rate
SDA	I2C Data

3 SOURCE FILES

3.1 Location of Source Files

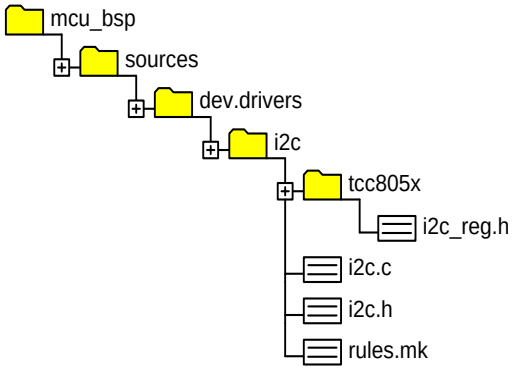

Figure 3.1 Location of Source Files

- Simple Description of Source Filesi2c_reg.c
  - I2C specific source code by chipset
- i2c_reg.h
  - I2C specific header by chipset
- i2c.c
  - I2C common source code
- i2c.h
  - I2C common header including API & DEFINE for users
- rules.mk
  - Make rules

4 ENUMERATIONS

4.1 I2CChannelState_t

Enumeration variables to manage the I2C transfer state

Declaration

```
typedef enum _I2CChannelState_
{
    I2C_STATE_IDLE        = 0,
    I2C_STATE_SEND        = 1,
    I2C_STATE_RECV        = 2,
    I2C_STATE_RECV_START  = 3,
    I2C_STATE_SEND_DONE   = 4,
    I2C_STATE_RECV_DONE   = 5,
    I2C_STATE_DISABLED    = 6,
    I2C_STATE_RUNNING     = 7
} I2CChannelState_t;
```

Description

Value	Description
I2C_STATE_IDLE	Index of I2C state: Idle
I2C_STATE_SEND	Index of I2C state: Send data
I2C_STATE_RECV	Index of I2C state: Wait for receiving data
I2C_STATE_RECV_START	Index of I2C state: Start to receive data
I2C_STATE_SEND_DONE	Index of I2C state: Complete sending data
I2C_STATE_RECV_DONE	Index of I2C state: Complete receiving data
I2C_STATE_DISABLED	Index of I2C state: Disable
I2C_STATE_RUNNING	Index of I2C state: Running (busy check)

Remark

None

5 CONSTANTS

5.1 I2C Base Address

Constant values for base address for each the I2C channel and the I2C configuration

Declaration

```
#define I2C_CH0_BASE      TCC_MICOM_I2C_BASE + 0x00000
#define I2C_CH1_BASE      TCC_MICOM_I2C_BASE + 0x10000
#define I2C_CH2_BASE      TCC_MICOM_I2C_BASE + 0x20000
#define I2C_PCFG_BASE     TCC_MICOM_I2C_BASE + 0x30000
#define I2C_CH0_VSLAVE    TCC_MICOM_I2C_BASE + 0x40000
#define I2C_CH1_VSLAVE    TCC_MICOM_I2C_BASE + 0x50000
#define I2C_CH2_VSLAVE    TCC_MICOM_I2C_BASE + 0x60000
```

Description

Value	*Description*
TCC_MICOM_I2C_BASE	*I2C controller register base address: 0x1B300000UL*
I2C_CH0_BASE	*I2C controller channel 0 base address: 0x1B300000UL*
I2C_CH1_BASE	*I2C controller channel 1 base address: 0x1B310000UL*
I2C_CH2_BASE	*I2C controller channel 2 base address: 0x1B320000UL*
I2C_PCFG_BASE	*I2C controller port configuration base address: 0x1B330000UL*
I2C_CH0_VSLAVE	*I2C controller virtual slave channel 0 base address: 0x1B340000UL*
I2C_CH1_VSLAVE	*I2C controller virtual slave channel 1 base address: 0x1B350000UL*
I2C_CH2_VSLAVE	*I2C controller virtual slave channel 2 base address: 0x1B360000UL*

Remark

Refer to "*TCC805x Full Specification*" about control registers offset in each base address. [1]

5.2 I2C Channel

Constant values for index of the I2C channels

Declaration

```
#define I2C_CH_MASTER0    (0UL)
#define I2C_CH_MASTER1    (1UL)
#define I2C_CH_MASTER2    (2UL)
```

Description

Value	*Description*
I2C_CH_MASTER0	*Index of I2C channel 0*
I2C_CH_MASTER1	*Index of I2C channel 1*
I2C_CH_MASTER2	*Index of I2C channel 2*

Remark

None

5.3 I2C Event

Constant values for I2C notification. When an I2C interrupt occurs, the I2C driver notifies event value, such as async transfer complete or receiving NACK, to user via a callback function.

Declaration

```
#define I2C_EVENT_XFER_COMPLETE  (0x00UL)
#define I2C_EVENT_XFER_ABORT     (0x01UL)
#define I2C_EVENT_NACK           (0x02UL)
```

Description

Value	*Description*
I2C_EVENT_XFER_COMPLETE	*Value to notify that transmitting I2C is done to user when transmitting async is completed*
I2C_EVENT_XFER_ABORT	*Value to notify that transmitting I2C is aborted*
I2C_EVENT_NACK	*Value to notify that I2C does not receive ACK from a slave device*

Remark

None

6 CALLBACK FUNCTIONS

6.1 I2CCallback

When async transfer is completed or an error interrupt occurs, the I2C driver notifies event information to user via a callback function.

Support events are as follows:

- Transmit done.
- Not receive ACK (=NACK).

Declaration

```
typedef void (*I2CCallback)
(
    Uint8   ucCh,
    int32   iEvent,
    void * pArg
);
```

Parameters

Value	Description
ucCh	Value of channel to be controlled
iEvent	Value of Event information. Refer to I2C Event.
pArg	Parameter from user

Return

None

Remark

None

7 STRUCTURES

7.1 I2CDev_t

Structure to store operating information of each I2C channel. I2CDev_t is declared in ***i2c_reg.h*** and has a default initial value. If you input parameter values when calling API such as ***I2C_Open***, the parameter values are automatically stored in this struct member variable.

Declaration

```
typedef struct _I2CDev_
{
    uint32              dClk;
    uint32              dBase;
    uint32              dPeriName;
    uint32              dIobusName;
    sint32              dSda;
    sint32              dScl;
    sint32              dPort;
    I2CCmdComplete_t    dComplete;
    I2CAsyncXfer_t      dAsync;
    I2CChannelState_t   dState;
} I2CDev_t;
```

Description

Value	Description
dClk	I2C channel operating speed (400 kHz/100 kHz)
dBase	I2C channel base address
dPeriName	Peri clock index
dIobusName	IOBUS index
dSda	I2C SDA GPIO index information
dScl	I2C SCL GPIO index information
dPort	Matched port number
dComplete	Struct to inform that transfer is completed
dAsync	Async transfer information
dState	I2C Channel state information

Remark

None

7.2 I2CAsyncXfer_t

Structure to store the I2C async transfer information. When you call ***I2C_Xfer*** with an async option, the I2C driver stores slave address and buffer information.

Declaration

```
typedef struct _I2CAsyncXfer_
{
    uint8      axSlaveAddr;
    uint8 *    axCmdBuf;
    uint8      axCmdLen;
    uint8 *    axOutBuf;
    uint8      axOutLen;
    uint8 *    axInBuf;
    uint8      axInLen;
    uint32     axOpt;
} I2CAsyncXfer_t;
```

Description

Value	Description
axSlaveAddr	Target slave address
axCmdBuf	Buffer of TX command data
axCmdLen	Length of TX command data
axOutBuf	Buffer of TX data
axOutLen	Length of TX data
axInBuf	Buffer of RX data
axInLen	Length of RX data
axOpt	Option (refer to I2C transfer options in i2c_reg.h). In async transmission, this option does nothing.

Remark

None

7.3 I2CCmdComplite_t

Structure to store callback function information to notify transfer complete to user. The callback function is called when an I2C interrupt occurs in async transfer.

Declaration

```
typedef struct _I2CCmdComplete_
{
    I2CCallback    ccCallBack;
    void *         ccArg;
} I2CCmdComplete_t;
```

Description

Value	Description
ccCallBack	Callback function pointer. Refer to the I2CCallback.
ccArg	Etc.

Remark

None

8 APIs

8.1 I2C_Init

Registers the I2C interrupt handler and enables I2C interrupt.

Declaration

```
void I2C_Init
(
    void
);
```

Parameters

None

Return

None

Remark

None

8.2 I2C_Deinit

Disables the I2C interrupt.

Declaration

```
void I2C_Deinit
(
    void
);
```

Parameters

None

Return

None

Remark

None

8.3 I2C_Open

Function to initialize the I2C Channel. Input valid parameter to operate the I2C channel.

- SCL/SDA values vary depending on the hardware construction.
- Select I2C clock speed 100 (kHz) or 400 (kHz).
- Register callback function to get notification from the I2C driver.

Declaration

```
SALRetCode_t I2C_Open
(
    uint8       ucCh,
    sint32      iGpioScl,
    sint32      iGpioSda,
    uint32      uiSpeedInKHz,
    I2CHandle   fnCallback,
    void *      pArg
);
```

Parameters

Value	Description
ucCh	Value of I2C channel
iGpioScl	Index of I2C SCL GPIO
iGpioSda	Index of I2C SDA GPIO
uiSpeedInKHz	I2C SCL speed
fnCallback	Callback function pointer
pArg	Etc.

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.4 I2C_Close

De-initializes the I2C channel. After calling this function, hardware resource is released and the I2C register values are cleared.

Declaration

```
SALRetCode_t I2C_Close
(
    uint8   ucCh
);
```

Parameters

Value	Description
ucCh	Value of channel to be controlled

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.5 I2C_Xfer

Function to transmit data to slave device. Before calling this function, the I2C channel must be initialized and configured.

Declaration

```
SALRetCode_t I2C_Xfer
(
    const uint8    ucCh,
    const uint8    ucSlaveAddr,
    const uint8    ucOutLen,
    uint8 *        ucpOutBuf,
    const uint8    ucInLen,
    uint8 *        ucpInBuf,
    uint32         uiOpt,
    uint8          ucAsync
);
```

Parameters

Value	Description
ucCh	Value of I2C channel
ucSlaveAddr	Specific slave address to be transferred
ucOutLen	Length of output data
ucpOutBuf	Buffer of output data
ucInLen	Length of input data (Length of read data)
ucpInBuf	Buffer of input data (Store data that is read after performing I2C READ operation.)
uiOpt	I2C transfer option
ucAsync	I2C transfer mode

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.6 I2C_XferCmd

Transmits the data with command. If this function is called, the I2C driver sends the command data first and then sends the remained data sequentially.

Declaration

```
SALRetCode_t I2C_XferCmd
(
    const uint8    ucCh,
    const uint8    ucSlaveAddr,
    const uint8    ucCmdLen,
    uint8 *        ucpCmdBuf,
    const uint8    ucOutLen,
    uint8 *        ucpOutBuf,
    const uint8    ucInLen,
    uint8 *        ucpInBuf,
    uint32         uiOpt,
    uint8          ucAsync
);
```

Parameters

Value	Description
ucCh	Value of channel to be controlled
ucSlaveAddr	Specific slave address to be transferred
ucCmdLen	Length of command data
ucpCmdBuf	Command buffer
ucOutLen	Length of output data
ucpOutBuf	Buffer of output data
ucInLen	Length of input data (Length of read data)
ucpInBuf	Buffer of input data (Store data that is read after performing I2C READ operation.)
uiOpt	I2C transfer option
ucAsync	I2C transfer mode

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.7 I2C_ScanSlave

Function to scan some slave device addresses that receive ACK for selected channel. By this function, you can get unknown slave address, but the information about the device that sends the ACK is unknown.

Declaration

```
uint32 I2C_ScanSlave
(
    uint8   ucCh
);
```

Parameters

Value	Description
ucCh	Value of channel to be controlled

Return

Value	Description
Slave address	Return one of detected slave addresses

Remark

None

9 How to Use I2C Driver

The I2C driver can transfer data to connected slave device. Before data transfer, call ***I2C_Init*** and ***I2C_Open*** with valid parameters. If I2C channel setup is successful without error, you can transfer data simply via calling ***I2C_Xfer***. Refer to ***i2c_test.c*** file to get more example code.

9.1 Operating I2C Channel

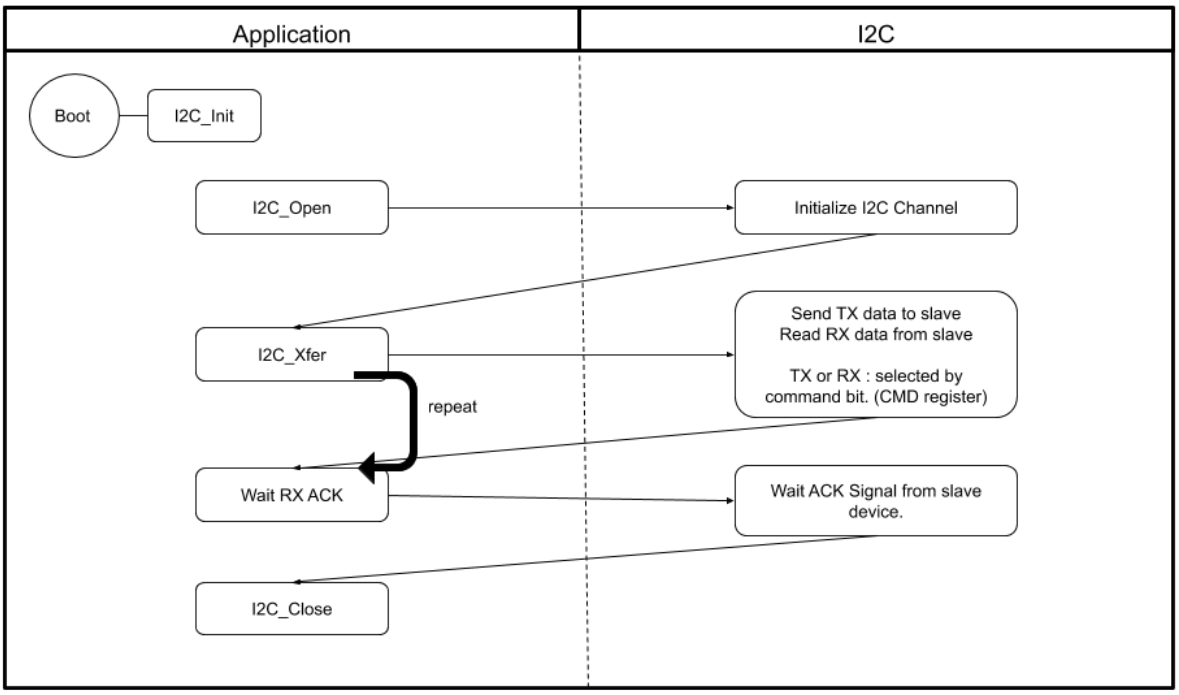

Figure 9.1 Diagram for I2C Driver

9.2 Example of Using I2C Channel

```
// tcc8050
#define I2C_TEST_SCL            (GPIO_GPMB(30UL))
#define I2C_TEST_SDA            (GPIO_GPMB(31UL))
#define I2C_TEST_CLK_RATE_100   (100)

{
    ucCh = 0;
    I2C_Open(ucCh, I2C_TEST_SCL, I2C_TEST_SDA, I2C_TEST_CLK_RATE_100, NULL , NULL);
    I2C_Xfer(…) or I2C_XferCmd(…);
    I2C_Close(ucCh);
}
```

9.3 Detection of Connected Slave Address

```
{
    // Enable debug log in I2C_ScanSlave function to see result on console.
    // Call I2C_ScanSlave(Channel) after initialize channel.

    // Result : Print all detected address of slave device.
                Xfer data to all address(0x00 ~ 0xff) and wait ACK.
                Receiving RX ACK means there is a slave device. But we don’t know what it is.

    // slave address maximum value : 0x7f (7bit)
}
```

10 REFERENCES

[1] TCC805x Full Specification

[2] Contact Telechips for more details: sales@telechips.com

11 REVISION HISTORY

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.
No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.
This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.
Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:
"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use other firmware of mp3:
"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use Digital Wave DRA solution:
"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:
"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:
"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:
"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:
"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."